

An Offline FIFO-Constrained MAPD Solver with Windowed PBS, and Hungarian Matching

Part of the Competition:

2025 Siemens Xcelerator Contest - MioVerse Track

2025 西门子Xcelerator公开赛MioVerse赛道 暨JCIOT开发者大赛

Participant: Jingming Chen 陈璟明 (University of Illinois Urbana-Champaign)*

1) Introduction

This solver brings together a **windowed PBS** in an **RHCR-style loop**, **orientation-aware actions**, **FIFO-aware Hungarian assignment**, and **multi-goal deliveries**—a combination that is not offered as a ready-to-use package in prior work. It respects my warehouse constraints while delivering strong makespan/throughput in practice.

2) Algorithmic architecture

2.1 High-level structure (RHCR-style windowing + PBS)

We use a **rolling horizon**: plan for a bounded window w , execute for h steps, then replan—resolving collisions only inside the next window. This mirrors Rolling-Horizon Collision Resolution (RHCR), which decomposes lifelong MAPF into repeated **Windowed MAPF** episodes with horizon w and replanning period h , yielding near-full-horizon throughput at a fraction of runtime.

Inside each episode, we run **Priority-Based Search (PBS)** on a **partial priority DAG** and re-order agents only where conflicts occur. Our windowed planner (wPBS), therefore, inherits the speed benefits reported for RHCR/PBS while accepting the usual incompleteness of bounded horizons.

*This work is completed while the author was a visiting student at Tsinghua University

2.2 Task assignment under FIFO: Hungarian matching on “front” jobs

To minimize dead-heading without violating FIFO, we expose **only the queue-front task per ramp** as eligible. At each replanning point, we solve a **rectangular Hungarian assignment** between free agents and these front tasks using the grid shortest-path distance as cost. The Hungarian algorithm is the classical min-cost matching method in polynomial time and is a standard building block in MAPD task assignment lines of work.

Concretely:

- A ramp is **locked** to a single agent until that agent **performs the pickup**.
- **Lock release on pickup** immediately exposes the **next** FIFO task at that ramp to the global matcher.
- We then recompute matching periodically or upon pickup/drop events.

2.3 Orientation-aware low-level

The low-level search state is $(x, y, orientation, t)$. Successors include **turns**, **forward move**, **wait**, **pickup**, **drop-off**; turns and (un)loading consume 1 timestep each. Collisions are checked on both vertices and **edge swaps** $(t, t+1)$. This extension differentiates the solver from standard 4-neighborhood MAPF where actions are limited to move/wait.

2.4 Multi-goal flexibility (four drop-offs)

At delivery, we evaluate all four orthogonal drop-off cells, choosing the least-cost feasible option under current window constraints—an instance of **multi-goal planning** that LNS-PBS/LNS-wPBS generalized in MAPD (MG-MAPD).

3) What is new vs. traditional baselines

Dimension	Traditional approach	My design
Coupling of assignment and paths	Two-stage (assign then plan), often greedy or single-task; or coupled (e.g., CBS-TA/ECBS-TA) that enumerates/updates assignments inside high-level search.	Windowed coupling: re-assign at episode boundaries and event triggers; use Hungarian matching only on FIFO-front tasks , then resolve collisions with wPBS.
FIFO at ramps	Often ignored or approximated; online methods (e.g., CENTRAL [1]) do not enforce FIFO stacks per ramp.	Strict per-ramp FIFO , but with lock-release-on-pickup so the next task can be pre-assigned immediately—reducing idle ramp time without violating FIFO.
Planning horizon	Full-horizon MAPF (slow) or per-step replanning; RHCR shows windowing preserves throughput while slashing runtime.	RHCR-style windows + PBS: resolves only near-term conflicts; faster replans; pliable plans that adapt to new assignments.
Motion model	Unit grid moves; orientation rarely modeled.	Orientation-aware kinematics with 90/180° turns & (un)loading actions costed at 1 timestep—closer to AGV reality.
Delivery cells	Single fixed goal per task.	Four admissible drop-offs , picked opportunistically each episode (MG-MAPD flavor).

Where it sits in the literature.

- **RHCR** [2] provides the windowed planning framework we adopt.
- **PBS** [3] supplies conflict resolution under partial orders; using it in a window produces the familiar speed/quality trade-off RHCR reports.
- **Task-assignment + path-finding coupling** is explored by **CBS-TA/ECBS-TA** [4], which show tighter coupling can reduce cost vs. naive “assign then plan.” My approach keeps the coupling lightweight (Hungarian between front tasks + wPBS) rather than embedding assignment into CBS nodes.
- **Offline planning with sequences** (TA-Hybrid [5]) uses TSP-style sequencing and “dummy paths” for completeness; my solver trades those heavy guarantees for speed and practical FIFO/ramp constraints.

4) Performance intuition

- **Hungarian on front tasks** lowers dead-heading globally compared to nearest-ramp greedy assignment; it keeps FIFO by construction and **pulls the next job forward as soon as pickup happens**.
- **Windowed PBS** reduces solve time per episode and, by frequent replans, opportunistically shortens paths when congestion clears—consistent with RHCR findings that bounded horizons preserve throughput while slashing runtime.

Complexity. Per assignment round: Hungarian $O(A^3)$ for A free agents ($|\text{front tasks}| \leq \text{number of active ramps}$). wPBS low-level expansions scale with window length w and branching at conflicts; shorter w typically gives dramatic runtime savings with small loss in solution quality per RHCR.

5) Validation and guarantees

- **Collisions:** The simulator rejects vertex and **edge-swap** conflicts by construction.
 - **FIFO:** Only the queue head is eligible at any time; **lock-release-on-pickup** preserves strict FIFO while boosting utilization.
 - **Completeness:** Like RHCR/wPBS, the planner is **incomplete** in the strict sense (bounded horizon). In practice, repeated re-planning and congestion relief work well on warehouse-dense maps, as reported for RHCR/PBS.
-

6) AIGC/LLM-assisted development workflow

I used LLMs (ChatGPT, Claude) end-to-end:

1. **Requirements capture** → **prompt engineering**. I converted the competition requirements (grid, ramps, FIFO, orientation, 4 drop-offs) into a precise prompt and added the instruction “*If anything is unclear, list assumptions*” to elicit clarifying questions. This surfaced edge cases early (e.g., turn costs, when to release FIFO locks) that helped me to iteratively improve my prompts, which was then used to generate code.

2. **Implementation acceleration.** I iterated with the LLM on clean module boundaries (assignment, planner, simulator), got **draft implementations** for the solver, and repeatedly asked the model to review edge-cases (edge conflicts, wait extension to cover the window, lock release on pickup).
3. **Report generation.** I again used AIGC to synthesize literature alignment, articulate novelty, and document trade-offs, cross-checking with the cited papers.

Why this matters: LLM co-design turned an ambitious integration (offline MAPD + FIFO + orientation + multi-goal + RHCR-style PBS + Hungarian) into a tractable project: faster ideation, consistent code scaffolding, and continuous critique using the model's knowledge of MAPD/MAPF literature. This **augmented** (not replaced) my engineering judgment and experiments, and produced a solution that is both **novel in combination** and **practically useful** for my warehouse variant.

List of chats with LLMs:

- Claude (Opus 4.1):
 - <https://claude.ai/share/9acefc74-49a1-4b92-871a-d80117f4b5b8>
 - <https://claude.ai/share/51ce5ba6-ec0f-4dbe-9129-a2536ceb586f>
 - ChatGPT (o3 and GPT-5):
 - <https://chatgpt.com/share/6899e945-e200-800e-8471-bcbcc9705a91>
 - <https://chatgpt.com/share/6899e97c-21a4-800e-9d51-0700be940e49>
-

7) Hyperparameters

- For my implementation, I chose $w=30$ and $h=15$ to produce my results. However, this can be changed to improve results or speed up computation. Start with $w \approx 20-30$, $h \approx 5-10$; shrink/expand w to trade runtime vs. replanning quality.

References

- [1] Ma, Hang, et al. "Lifelong multi-agent path finding for online pickup and delivery tasks." *arXiv preprint arXiv:1705.10868* (2017).
- [2] Li, Jiaoyang, et al. "Lifelong multi-agent path finding in large-scale warehouses." *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. No. 13. 2021.
- [3] Xu, Qinghong, et al. "Multi-goal multi-agent pickup and delivery." *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022.
- [4] Hönl, Wolfgang, et al. "Conflict-based search with optimal task assignment." *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems*. 2018.
- [5] Liu, Minghua, et al. "Task and path planning for multi-agent pickup and delivery." *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. 2019.

Appendix

I. Prompt used to iteratively refine the final prompt, and then used to generate code.

Task

Your task is to design and fully implement a Python program that solves the entire problem detailed below. The problem is a variant of MAPF (multi-agent path finding) and MAPD (multi-agent pickup and delivery).

Our version of the MAPD problem is a variant that models AGVs (automated guided vehicles) in a 20x20 warehouse, and it is different from traditional MAPD problems in the following ways:

- * Our problem is offline, meaning that all tasks are already known at the start, and no additional tasks are added during runtime. (Though, online MAPD algorithms would work for offline versions too.)

- * Instead of moving to adjacent four cells or waiting like in traditional MAPD problems, our agent's actions consist of [move forward, turn 90 or 180 degrees, wait, pick up, and drop off].
- * Our problem has four available goal locations to choose from for each task, instead of a single goal like in traditional MAPD problems.

You should draw inspiration from the provided papers (MAPD, ECBS-TA, RHCR, LNS-wPBS, etc.) when designing a solution for this problem. The final algorithm should satisfy all constraints of our problem, generate paths that are optimal or approximately optimal, and be computationally efficient (use the windowing approach outlined in the RHCR paper).

Details

Environment

Aspect	Description
-----	-----
Warehouse dimensions	A grid of 20 (X) × 20 (Y) cells. Coordinates: (1,1) bottom-left; (20,20) top-right. Movement only along grid centers; no diagonals.
Loading Ramps	Located on warehouse walls. Each has one pick-up point: the adjacent interior cell (e.g., ramp at (1,6) → pick-up at (2,6)). Parcels arrive here and form a stack, and parcels can only be picked up in FIFO order (eg. earlier rows in task input file for a given start-point must be picked up first).
Offload Ramps	Labeled by destination city (e.g., Beijing, Shenzhen, etc.). A parcel can be dropped off on any of the four orthogonally adjacent cells (above, below, left, right) to its designated offload ramp cell. AGVs are free to choose whichever of the four is reachable or least congested.
AGV orientation	The orientation of each AGV must be considered during path planning. 0 degrees faces the east, 90 degrees faces the north, 180 degrees faces the west, and 270 degrees faces the south.
Obstacles	AGVs must remain within the 20x20 warehouse, and the loading/offload ramps are treated as obstacles (eg. AGVs cannot drive "through" the ramps).

AGVs

Attribute	Specification
-----	-----
Size	Occupies exactly 1 grid cell.

| Movement | AGVs can move forward in the direction they face at 1 cell per timestep (no need to model acceleration/deceleration). |

| Turning | AGVs cannot move to another cell when turning; a turn (90°, 180°, or 270°) takes 1 timestep. |

| Orientation Angles | 0° = +X, 90° = +Y, 180° = -X, 270° = -Y. Initial coordinates and orientations for each AGV are given. |

| Carrying Capacity | 1 parcel at a time. |

| Waiting | AGVs may wait in place for any integer number of timesteps. |

|

| Loading/Unloading | AGVs must be stationary during pick-up or drop-off (no turning or moving). Pick-up or drop-off takes 1 timestep each. |

| Collisions | Collisions must be strictly avoided during the path planning phase. A collision occurs if (a) two or more AGVs occupy the same cell at the same time, or (b) two AGVs simultaneously swap cells. |

Program input and outputs

Input files (located under the same directory):

* map_data.csv

* This file consists of the locations of all the pick-up points (marked start_point) and drop-off points (marked end_point), and the initial locations and orientation (pitch) of the AGVs.

* This file is formatted as:

```
type,name,x,y,pitch
start_point,Tiger,1,6,
start_point,Dragon,1,10,
...
end_point,Beijing,6,4,
end_point,Shanghai,6,8,
...
agv,Optimus,3,1,90
agv,Bumblebee,6,1,90
...
```

* task_csv.csv

* This file consists of all the parcels (tasks) to be delivered from the pick-up points to the drop-off points, whether the parcels are marked as Urgent, and the maximum timestep that the parcel has to be delivered on.

* This file is formatted as:

```
task_id,start_point,end_point,priority,remaining_time
Tiger-1,Tiger,Xiamen,Normal,None
Tiger-2,Tiger,Changsha,Normal,None
...
Monkey-1,Monkey,Shanghai,Normal,None
```


Monkey-2,Monkey,Chengdu,Normal,None
Monkey-3,Monkey,Beijing,Urgent,45
...

Output file (also under the same directory):

* agv_trajectory.csv

* This file consists of the position, angle, status, and other related attributes of every AGV at each timestep.

* The location, orientation, status, and other attributes of every AGV is recorded for every single timestep, even if an AGV is stationary or has no assigned task.

* This file is formatted as (this is an example output; the actual generated paths do not need to follow this example output):

timestamp,name,X,Y,pitch,loaded,destination,Emergency,task-id

0,Bluestreak,15,20,270,false,,false,

0,Bumblebee,6,1,90,false,,false,

...

410,Bluestreak,5,12,90,true,Suzhou,false,

410,Bumblebee,7,16,90,false,,false,

...

411,Bluestreak,5,12,90,false,,false,Rabbit-20

411,Bumblebee,7,16,90,false,,false,

...

* The task-id field only needs to have a value during pick-up or drop-off timesteps. It can be empty for any other actions.

Implementation details

* You should implement a task assignment and multi-agent path finding method similar to ECBS-TA or LNS-wPBS with a windowing approach from RHCR to ensure that it is computationally efficient.

* It is up to you to decide whether the program should assign all tasks up front (producing long individual task sequences for each AGV) or assign only one task at a time, then re-plan after an AGV finishes a delivery. Please review the details in the papers to determine which one is the most suitable.

* The multi-agent and collision-free path-finding part should consider the orientation of the AGVs during its search, meaning that it should account for the 1 timestep needed to change AGV orientation and the 1 timestep needed to pick up or drop off. This means that to ensure an optimal collision-free and lowest cost path for each AGV, the algorithm should create paths that do not perform unnecessary turns.

* For algorithms that require a "parking location" or "non-task endpoints" for AGVs, use the initial locations of AGVs outlined in map_data.csv as these designated locations.

- * Some tasks are marked as "Urgent" in task_csv.csv and have a time limit. For our purposes, we ignore this attribute and the time limit and treat all tasks as equals. However, the output file should still mark Urgent tasks with "true" in the "Emergency" column.
- * The fully implemented program should be modular if possible, meaning that the task assignment, path planning, or other major parts of the program can be swapped for any other algorithms or implementations if wanted.
- * Do NOT simplify or skip any parts of the algorithm during implementation, please implement the complete algorithms (ex., for CBS, rolling horizon, or path-finding algorithms).
- * The scale of this problem: there will be 100 tasks and 12 AGVs, but the number of tasks or agents may be increased in testing.
- * The goal is to create paths for every AGV that result in the smallest makespan (the number of timesteps it takes from start to when all tasks are completed) while keeping the program computationally efficient.
- * The problem is solved once every task has been completed. The simulation can stop once the last parcel is dropped off; it does not need to wait until the last AGV returns to its parking location.

Your thinking should be thorough and so it's fine if it's very long. You can think step by step before and after each action you decide to take.

You MUST iterate and keep going until the problem is solved.

Only terminate your turn when you are sure that the problem is solved. Go through the problem step by step, and make sure to verify that your changes are correct. NEVER end your turn without having solved the problem, and when you say you are going to make a tool call, make sure you ACTUALLY make the tool call, instead of ending your turn.

If you are unclear or need to make assumptions about any aspect of this task, please list them in your answer so that I can provide details to clarify them for you.

Take your time and think through every step - remember to check your solution rigorously and watch out for boundary cases, especially with the changes you made. Your solution must be perfect. If not, continue working on it. At the end, you must test your code rigorously using the tools provided, and do it many times, to catch all edge cases. If it is not robust, iterate more and make it perfect. Failing to test your code sufficiently rigorously is the NUMBER ONE failure mode on these types of tasks; make sure you handle all edge cases.

You MUST plan extensively before each function call, and reflect extensively on the outcomes of the previous function calls. DO NOT do this entire process by making function calls only, as this can impair your ability to solve the problem and think insightfully.